

SE 423 Mechatronics

Homework Assignment #Code Commenting

All homework code is expected to be **commented** and have your **initialed** on each of the lines. Points WILL be taken off if this is not done.

Exercise 1: Internal commenting

Below are examples of how we are expecting you to comment your code. **MAKE SURE YOU DO THIS!!!** Also make sure you put in the **top of the file** what the initials to be checked for are!

```
// Andy Jones is commenting this Code. Comments marked by my initials AAJ
//#####
// FILE: LABstarter_main.c
//
// TITLE: Lab Starter
//#####
#include <stdio.h>
#include <stdlib.h>
.... Omitted Code
.... You should not omit code in your submission, it has just been removed here for the sake of conciseness
....
uint32_t numTimer0calls = 0;
uint32_t numSWIcalls = 0;
extern uint32_t numRXA;
uint16_t UARTPrint = 0;
uint16_t LEDdisplaynum = 0;
float Kp = 3.4; // AAJ Kp and Ki gains used for the PI controller. Hand tuned for desired response.
float Ki = 29.3; // I did find that a number of gain combinations worked, but settled on these gains.
float Vref = 2.5;
float Vactual = 0;
float Ik = 0;
float Vout = 0;
void main(void)
{
    // PLL, WatchDog, enable Peripheral Clocks
    // This example function is found in the F2837xD_SysCtrl.c file.
    InitSysCtrl();
    .... Omitted Code
    .... You should not omit code in your submission, it has just been removed here for the sake of conciseness
    ....
    // Configure CPU-Timer 0, 1, and 2 to interrupt every second:
    // 200MHz CPU Freq, 1 second Period (in uSeconds)
    ConfigCpuTimer(&CpuTimer0, 200, 5000); //AAJ Set CPU Timer 0 to call its interrupt function every 5ms. This is the spe
    ConfigCpuTimer(&CpuTimer1, 200, 20000);
    ConfigCpuTimer(&CpuTimer2, 200, 40000);
    .... Omitted Code
    .... You should not omit code in your submission, it has just been removed here for the sake of conciseness
    ....
    // Enable global Interrupts and higher priority real-time debug events
    EINT; // Enable Global interrupt INTM
    ERTM; // Enable Global realtime interrupt DBGM

    !!!! There is a second page !!!!
    // IDLE loop. Just sit and loop forever
```

```

while(1)
{
    if (UARTPrint == 1 ) {
        // AAJ Here I am using GPIO67 to see how much time the serial_printf function takes to run. In lab I will connect
        // to GPIO67's pin and measure the width of the pulse created by setting GPIO67 high (3.3Volts) before the function
        // setting GPIO67 low (0Volts) after the function is completed. GPIO67 is controlled by the GPC registers. GPIO67 is
        // in the GPC registers. Bit 0 is for GPIO64 Bit 1 is for GPIO65, Bit 2 is for GPIO66, Bit 3 is for GPIO67, Bit 4 is for GPIO68,
        // The below bitfield C statement hides the bit number from me and allows me to just set the associated bit per
        // Setting a bit to 1 in the GPCSET register causes that associated output pin to be high (3.3 Volts).
        // Setting a bit to 0 in the GPCSET register does nothing.
        GpioDataRegs.GPCSET.bit.GPIO67 = 1;
        //AAJ Printing Desired and Actual Speed %.3f tells printf to print 3 decimal places of float variable
        serial_printf(&SerialA, "%.3f Desired Speed, %.3f Actual Speed\r\n", Vref, Vactual);
        UARTPrint = 0;
        // AAJ Set GPIO67 to low (0Volts) so Oscilloscope and see when serial_printf function is complete
        // Setting a bit to 1 in the GPCCLEAR register causes that associated output pin to be low (0 Volts). Setting a bit
        // to 0 in the GPCCLEAR register does nothing.
        GpioDataRegs.GPCCLEAR.bit.GPIO67 = 1
    }
}

// cpu_timer0_isr - CPU Timer0 ISR
__interrupt void cpu_timer0_isr(void)
{
    Vactual = readfeedback(); // AAJ this is a dummy function for this example.
    // AAJ Here I am calculating the PI control law. I am using the "forward rule" to approximate the integral
    // Then I calculate the control effort Vout which is Kp times the error between desired speed and actual speed
    // and Ki times the integral approximation of the error. I also check for integral windup and then saturate
    // the output between -10 and +10.
    Ik = Ikold + (Vref-Vactual)*0.005;
    Vout = Kp*(Vref-Vactual) + Ki*Ik;
    if (fabs(Vout) > 10) Ik = Ikold;
    if (Vout > 10) Vout = 10;
    if (Vout < -10) Vout = -10;
    outPWM(Vout); //AAJ dummy function to output control
    Ikold = Ik; //AAJ save past state for next 5ms
    CpuTimer0.InterruptCount++;
    numTimer0calls++;
    if ((numTimer0calls%20) == 0) {
        UARTPrint = 1; //AAJ Print to Tera Term every 100ms 5ms * 20 = 100ms
        // Blink LaunchPad Red LED
        GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1;
    }

    // Acknowledge this interrupt to receive more interrupts from group 1
    PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
}

... Omitted Code
... You should not omit code in your submission, it has just been removed here for the sake of conciseness
...

```

Exercise 2: Function documentation

When you specifically create a function (you do not need to do this for the provided interrupts or the main function, but you are welcome to) you should be using [Doxygen JAVADOC](#) styling for function documentation (it is what the industry uses so use it!).

Each function should have the following structured documentation:

1. Brief summary (first sentence)
2. A more detailed description.
3. `@param <parameter name> <description of parameter>` for each parameter, ordered by when it appears in the function's parameters
4. `@return <description of what gets returned>`, when applicable (`void` functions would not have this)
5. `@pre <parameter expectations>`, providing expectations of what the input is supposed to satisfy when inputted into it
6. `@post <return expectations>`, providing expectations of how the parameters are modified during the function
7. `@note` or `@warning` when relevant

```
/**
 * AAJ: A brief history of JavaDoc-style (C-style) comments.
 *
 * This is the typical JavaDoc-style C-style comment. It starts with two
 * asterisks.
 *
 * @param theory Even if there is only one possible unified theory. it is just a
 *               set of rules and equations.
 */
void cstyle( int theory );
```

```
/**
 * AAJ: Computes the arithmetic mean of an array of doubles.
 *
 * @param data Pointer to the first element of the array.
 * @param length Number of elements in the array (must be > 0).
 * @return The arithmetic mean of the values in the array.
 *
 * @pre data != NULL
 * @pre length > 0
 * @post The input array is not modified.
 */
double mean(const double *data, size_t length);
```

```
/**
 * AAJ: Opens a file and reads its contents into a buffer.
 *
```

```

* The function allocates memory dynamically for the buffer.
* The caller is responsible for freeing it with free().
*
* @param path Null-terminated string specifying the file path.
* @param buffer Output pointer that will receive allocated memory.
* @param size Output parameter storing the number of bytes read.
*
* @return 0 on success, negative error code on failure.
*
* @retval 0 Success.
* @retval -1 File could not be opened.
* @retval -2 Memory allocation failed.
*
* @pre path != NULL
* @pre buffer != NULL
* @pre size != NULL
* @post On success, *buffer points to valid allocated memory.
*
* @warning The function does not append a null terminator.
*/
int read_file(const char *path, unsigned char **buffer, size_t *size);

```

```

/**
* AJ: Initializes the ADC peripheral.
*
* Configures hardware registers and enables interrupts required
* for analog-to-digital conversion.
*
* @param config Pointer to ADC configuration structure.
*
* @return true if initialization succeeds, false otherwise.
*
* @pre config != NULL
* @post ADC hardware is ready for sampling.
*
* @note This function must be called before adc_read().
*/
bool adc_init(const adc_config_t *config);

```

Exercise 3: Function template

Below is a template that you can use,

```

/**
* AJ: Brief one-line summary.
*
* Detailed description if needed.
*
* @param name Description.
* @return Description.
*
* @pre Conditions that must hold.
* @post Guarantees after execution.
*/

```
